LAKIREDDY BALI REDDY COLLEGE OF ENGINEERING (AUTONOMOUS)
Department of Computer Science and Engineering | Full Stack Development Lab

STUDENT PLACEMENT PLANNER — MAIN SOURCE CODE & WORKFLOW COMPENDIUM

Student: V. Vijay Kumar (24761A0559) • CSE Batch 2024–2028 • Project Guide: Dr. K. DeviPriya, Associate Professor

1. Multi-Tier Enterprise Architecture & Technology Integration

The Student Placement Planner is architected as an enterprise MVC web application. The presentation tier uses dynamic JSP pages with unified fragments (_header.jspf, _footer.jspf), custom CSS design tokens, and vanilla JavaScript. Requests are routed through Java Servlets with session verification via AuthFilter. The business service layer processes intelligence calculations, while the DAO persistence layer communicates with Oracle Database XE via JDBC DBUtil.

Tier	Primary Modules & Files	Runtime Responsibilities & Workflow Role
1. Frontend	index.jsp, views/*.jsp, style.css, app.js	Responsive UI, dark/light theme switching, live multi-filter search, modal forms, inline status updates
2. Web Controllers	AuthServlet, DashboardServlet, JobServlet, AdminJobServlet, AuthFilter	HTTPServlet routing, form parameter parsing, session validation, authentication tokens, admin RBAC guards
3. Service Layer	PlacementIntelligenceService, ResumeService, JobDiscoveryService	4-pillar readiness score calculation, resume text extraction & ATS audit, genuine career catalog aggregation
4. DAO & Database	DBUtil, StudentDAO, JobDAO, SkillDAO, TaskDAO, ApplicationDAO	Bounded connection pool, Oracle 10g/21c DDL/DML, sequence generation, ORA-12516 retry backoff resilience

2. Oracle Relational Database DDL Schema & Referential Constraints

Normalized tables in Oracle Database XE maintain academic candidates, job drives, technical skills, preparation milestones, and job applications with cascading delete integrity and unique constraint protections.

Relational Schema DDL & Referential Integrity — database/oracle_schema.sql [SQL]

```
-- database/oracle_schema.sql (Core Relational Schema & Constraints)
CREATE TABLE students (
    id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    name VARCHAR2(100) NOT NULL, email VARCHAR2(150) UNIQUE NOT NULL, password_hash VARCHAR2(64) NOT NULL,
    role VARCHAR2(20) DEFAULT 'STUDENT' NOT NULL, department VARCHAR2(60) DEFAULT 'CSE',
    year_of_study NUMBER(2) DEFAULT 4, cgpa NUMBER(3,2) DEFAULT 7.5, created_at DATE DEFAULT SYSDATE NOT NULL,
    CONSTRAINT chk_student_role CHECK (role IN ('STUDENT', 'ADMIN'))
);
CREATE TABLE jobs (
    id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    title VARCHAR2(150) NOT NULL, company VARCHAR2(150) NOT NULL, location VARCHAR2(100),
    description VARCHAR2(1000), eligibility VARCHAR2(150), min_cgpa NUMBER(3,2) DEFAULT 0.0,
    deadline DATE, created_at DATE DEFAULT SYSDATE NOT NULL
);
CREATE TABLE applications (
    id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    student_id NUMBER NOT NULL REFERENCES students(id) ON DELETE CASCADE,
    job_id NUMBER NOT NULL REFERENCES jobs(id) ON DELETE CASCADE,
    status VARCHAR2(30) DEFAULT 'APPLIED' NOT NULL, applied_at DATE DEFAULT SYSDATE NOT NULL,
    CONSTRAINT uq_student_job UNIQUE (student_id, job_id)
);
```

3. High-Reliability Connection Pooling & ORA-12516 Resilience

DBUtil.java provides a thread-safe connection pool with dynamic proxy interception and exponential retry backoff to prevent resource exhaustion and transient listener busy states in Oracle Database XE.

Oracle JDBC Connection Pooling & Retry Backoff — src/main/java/com/placementplanner/util/DBUtil.java [JAVA]

```
// src/main/java/com/placementplanner/util/DBUtil.java (Connection Pool & Retry)
public final class DBUtil {
    private static final BlockingQueue<Connection> POOL;
    private static final int MAX_POOL_SIZE = 5;
    static {
        POOL = new ArrayBlockingQueue<>(MAX_POOL_SIZE);
        Runtime.getRuntime().addShutdownHook(new Thread(DBUtil::closePool, "DBUtil-Shutdown-Hook"));
    }
    public static Connection getConnection() throws SQLException {
        ensureDriverLoaded();
        return POOL_ENABLED ? borrowPooledConnection() : createPhysicalConnectionWithRetry();
    }
    private static Connection createPhysicalConnectionWithRetry() throws SQLException {
        for (int attempt = 1; attempt <= 3; attempt++) {
            try {
                return DriverManager.getConnection(getUrl(), getUser(), getPassword());
            } catch (SQLException e) {
                boolean isBusy = e.getErrorCode() == 12516 || e.getMessage().contains("12516");
                if (isBusy && attempt < 3) {
                    try { Thread.sleep(300L * attempt); } catch (InterruptedException ie) {}
                } else throw e;
            }
        }
        throw new SQLException("Oracle database connection failed after 3 attempts.");
    }
}
```

2. User Authentication, Cryptographic Security & Route Protection

The security subsystem implements industry-standard credential protection. Passwords are encrypted with SHA-256 via PasswordUtil. AuthServlet verifies credentials against the database hash, invalidates stale sessions to mitigate session fixation attacks, and binds the authenticated Student model into the HTTP session. AuthFilter intercepts requests across all protected routes, denying unauthenticated access and restricting /admin/* endpoints strictly to administrators.

SHA-256 Cryptographic Password Hashing src/main/java/com/placementplanner/util/PasswordUtil.java [JAVA]

```
// src/main/java/com/placementplanner/util/PasswordUtil.java
public static String hash(String value) {
    if (value == null) return "";
    try {
        MessageDigest md = MessageDigest.getInstance("SHA-256");
        byte[] hash = md.digest(value.getBytes(StandardCharsets.UTF_8));
        return HexFormat.of().formatHex(hash);
    } catch (NoSuchAlgorithmException e) { throw new IllegalStateException(e); }
}
```

Student Registration & Authentication DAO src/main/java/com/placementplanner/dao/StudentDAO.java [JAVA]

```
// src/main/java/com/placementplanner/dao/StudentDAO.java (Registration & Authentication)
public void register(String name, String email, String password) throws SQLException {
    if (emailExists(email)) throw new SQLException("Account with email already exists.");
    String sql = "INSERT INTO students (student_id, full_name, name, email, password_hash, role, created_at) "
               + "VALUES (STUDENTS_SEQ.NEXTVAL, ?, ?, ?, ?, 'STUDENT', SYSDATE)";
    try (Connection con = DBUtil.getConnection(); PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setString(1, name.trim()); ps.setString(2, name.trim());
        ps.setString(3, email.trim().toLowerCase()); ps.setString(4, PasswordUtil.hash(password));
        ps.executeUpdate();
    }
}
public Student authenticate(String email, String password) throws SQLException {
    String sql = "SELECT * FROM students WHERE LOWER(TRIM(email)) = LOWER(TRIM(?))";
    try (Connection con = DBUtil.getConnection(); PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setString(1, email.trim());
        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) {
                String dbHash = rs.getString("password_hash");
                String inputHash = PasswordUtil.hash(password);
                if (dbHash != null && (dbHash.equalsIgnoreCase(inputHash) || dbHash.equals(password)))
                    return mapStudentRow(rs);
            }
        }
    }
    return null;
}
```

Global Security Interceptor & Admin RBAC Guard src/main/java/com/placementplanner/web/AuthFilter.java [JAVA]

```
// src/main/java/com/placementplanner/web/AuthFilter.java (Global Route Interception)
@WebFilter(urlPatterns = {"/dashboard", "/profile", "/skills", "/tasks", "/apply", "/admin/*"})
public class AuthFilter implements Filter {
    public void doFilter(ServletRequest req, ServletResponse res, FilterChain chain) throws IOException, ServletException {
        HttpServletRequest request = (HttpServletRequest) req;
        HttpSession session = request.getSession(false);
        Student student = (session != null) ? (Student) session.getAttribute("student") : null;
        if (student == null) {
            ((HttpServletResponse) res).sendRedirect(request.getContextPath() + "/login");
            return;
        }
        if (request.getServletPath().startsWith("/admin") && !student.isAdmin()) {
            ((HttpServletResponse) res).sendError(HttpServletResponse.SC_FORBIDDEN, "Administrator access required.");
            return;
        }
        chain.doFilter(req, res);
    }
}
```

Authentication Session Lifecycle Controller src/main/java/com/placementplanner/web/AuthServlet.java [JAVA]

```
// src/main/java/com/placementplanner/web/AuthServlet.java (Session Lifecycle Controller)
if ("/login".equals(path)) {
    Student student = dao.authenticate(email.trim(), password);
    if (student == null) {
        request.setAttribute("error", "Invalid email address or password. Please try again.");
        request.getRequestDispatcher("/WEB-INF/views/login.jsp").forward(request, response);
        return;
    }
    HttpSession oldSession = request.getSession(false);
    if (oldSession != null) oldSession.invalidate(); // Prevent session fixation
    HttpSession session = request.getSession(true);
    session.setAttribute("student", student);
    response.sendRedirect(request.getContextPath() + "/dashboard");
}
```

3. Student Career Command Center & Placement Readiness Intelligence

Upon authentication, DashboardServlet fetches candidate profile credentials, active applications, confirmed technical skills, and scheduled preparation milestones using a single leased database connection. It invokes PlacementIntelligenceService to compute an auditable, transparent 4-pillar readiness score (0–100%) and calculate job-specific skill matching percentages with 1-click learning milestone recommendations.

Dashboard Controller & Data Aggregation — src/main/java/com/placementplanner/web/DashboardServlet.java [JAVA]

```
// src/main/java/com/placementplanner/web/DashboardServlet.java (Aggregated Data Retrieval)
@WebServlet("/dashboard")
public class DashboardServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
        HttpSession session = request.getSession(false);
        Student student = (Student) session.getAttribute("student");
        long studentId = student.getId();

        try (Connection con = DBUtil.getConnection()) {
            List<Map<String, Object>> allJobs = JobDiscoveryService.getAllOpportunities(con);
            List<Map<String, Object>> skills = skillDAO.findByStudent(con, studentId);
            List<Map<String, Object>> tasks = taskDAO.findByStudent(con, studentId);
            Set<Long> appliedJobIds = applicationDAO.getAppliedJobIds(con, studentId);
            ResumeInfo resumeInfo = ResumeService.getResumeInfo(studentId);

            ReadinessReport report = PlacementIntelligenceService.generateReadinessReport(
                student, skills, resumeInfo, tasks, allJobs
            );
            int completed = (int) tasks.stream().filter(t -> "Completed".equalsIgnoreCase((String) t.get("status"))).count();
            request.setAttribute("readinessReport", report);
            request.setAttribute("totalJobs", allJobs.size());
            request.setAttribute("appliedCount", appliedJobIds.size());
            request.setAttribute("progressPercent", tasks.isEmpty() ? 0 : (completed * 100 / tasks.size()));
            request.getRequestDispatcher("/WEB-INF/views/dashboard.jsp").forward(request, response);
        }
    }
}
```

Auditable 4-Pillar Placement Readiness Engine — src/main/java/com/placementplanner/service/PlacementIntelligenceService.java [JAVA]

```
// src/main/java/com/placementplanner/service/PlacementIntelligenceService.java (4-Pillar Formula)
public static ReadinessReport generateReadinessReport(Student student, List<Map<String, Object>> skillsList,
        ResumeInfo resumeInfo, List<Map<String, Object>> tasksList, List<Map<String, Object>> allJobs) {
    // Pillar 1: Academic Eligibility (Max 25%) -> Branch (8%) + Year (7%) + CGPA (10%)
    int academic = 0;
    if (student != null) {
        if (student.getDepartment() != null && !student.getDepartment().isBlank()) academic += 8;
        if (student.getYearOfStudy() > 0) academic += 7;
        if (student.getCgpa() != null && student.getCgpa() > 0.0) academic += 10;
    }
    // Pillar 2: Technical Skills Portfolio (Max 25%) -> 5% per confirmed skill (capped at 5 skills)
    int skillsScore = Math.min(25, skillsList.size() * 5);

    // Pillar 3: Resume Completeness & ATS Screening (Max 25%) -> Scaled from ATS score
    int resumeScore = (resumeInfo != null && resumeInfo.isUploaded())
                    ? (int) Math.round((resumeInfo.getScore() / 100.0) * 25.0) : 0;

    // Pillar 4: Milestone Task Execution (Max 25%) -> Completed tasks / Total tasks ratio
    int totalTasks = tasksList != null ? tasksList.size() : 0;
    int completed = (int) tasksList.stream().filter(t -> "Completed".equalsIgnoreCase((String) t.get("status"))).count();
    int tasksScore = totalTasks > 0 ? (int) Math.round((completed * 25.0) / totalTasks) : 0;

    int overall = Math.min(100, academic + skillsScore + resumeScore + tasksScore);
    return new ReadinessReport(overall, academic, skillsScore, resumeScore, tasksScore, ...);
}
```

Job Skill Matching & 1-Click Actionable Learning Plan — src/main/java/com/placementplanner/service/PlacementIntelligenceService.java [JAVA]

```
// Job Skill Matching Engine (PlacementIntelligenceService.java)
public static JobMatchResult evaluateJobMatch(Map<String, Object> job, List<String> studentSkills, Set<String> resumeSkills) {
    List<String> required = parseSkillList(job.get("requiredSkills"));
    List<String> matchedReq = new ArrayList<>(), missingReq = new ArrayList<>();
    for (String req : required) {
        if (isSkillMatched(req, studentSkills)) matchedReq.add(req);
        else missingReq.add(req);
    }
    int matchPercent = required.isEmpty() ? 100 : (int) Math.round((matchedReq.size() * 100.0) / required.size());
    List<RecommendedTask> tasks = new ArrayList<>();
    for (String missing : missingReq) tasks.addAll(generateTasksForSkill(missing));
    return new JobMatchResult(jobId, job, required, matchedReq, missingReq, matchPercent, tasks);
}
```

4. Campus Placement Drives, External Portal Tracking & Admin Console

The job discovery subsystem combines internal campus placement opportunities stored in Oracle with verified enterprise openings via JobDiscoveryService. Students submit internal applications or record external employer portal submissions. Administrators access AdminJobServlet to publish recruitment drives and manage candidate hiring stages (Applied, Shortlisted, Interview, Offered, Placed).

Job Drive Persistence & Company Normalization src/main/java/com/placementplanner/dao/JobDAO.java [JAVA]

```
// src/main/java/com/placementplanner/dao/JobDAO.java (Job Catalog & Drive Publishing)
public List<Map<String, Object>> findAll(Connection con) throws SQLException {
    String sql = "SELECT j.*, c.company_name FROM jobs j LEFT JOIN companies c ON j.company_id = c.company_id ORDER BY 1 DESC";
    try (PreparedStatement ps = con.prepareStatement(sql); ResultSet rs = ps.executeQuery()) {
        List<Map<String, Object>> jobs = new ArrayList<>();
        while (rs.next()) jobs.add(mapJobRow(rs));
        return jobs;
    }
}
public void add(String title, String company, String location, String description,
                String eligibility, Double minCgpa, String deadline) throws SQLException {
    try (Connection con = DBUtil.getConnection()) {
        long companyId = findOrCreateCompany(con, company, location);
        String sql = "INSERT INTO jobs (job_id, company_id, job_title, location, description, "
                   + "eligibility, min_cgpa, deadline, created_at) "
                   + "VALUES (JOBS_SEQ.NEXTVAL, ?, ?, ?, ?, ?, ?, ?, SYSDATE)";
        try (PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setLong(1, companyId); ps.setString(2, title.trim()); ps.setString(3, location);
            ps.setString(4, description); ps.setString(5, eligibility); ps.setDouble(6, minCgpa != null ? minCgpa : 0.0);
            ps.setDate(7, parseDate(deadline));
            ps.executeUpdate();
        }
    }
}
```

Application Submission & Status Progression DAO src/main/java/com/placementplanner/dao/ApplicationDAO.java [JAVA]

```
// src/main/java/com/placementplanner/dao/ApplicationDAO.java (Submission & Status Update)
public boolean apply(long studentId, long jobId) throws SQLException {
    if (hasApplied(studentId, jobId)) return false;
    String sql = "INSERT INTO applications (application_id, student_id, job_id, status, application_date) "
               + "VALUES (APPLICATIONS_SEQ.NEXTVAL, ?, ?, 'APPLIED', SYSDATE)";
    try (Connection con = DBUtil.getConnection(); PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setLong(1, studentId); ps.setLong(2, jobId);
        ps.executeUpdate();
        return true;
    } catch (SQLException e) {
        if (e.getErrorCode() == 1) return false; // ORA-00001 Unique constraint
        throw e;
    }
}
public boolean updateStatus(long applicationId, String newStatus) throws SQLException {
    String sql = "UPDATE applications SET status = ? WHERE application_id = ?";
    try (Connection con = DBUtil.getConnection(); PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setString(1, newStatus.trim().toUpperCase()); ps.setLong(2, applicationId);
        return ps.executeUpdate() > 0;
    }
}
```

Administrator Drive Publisher & Applicant Workflow Servlet src/main/java/com/placementplanner/web/AdminJobServlet.java [JAVA]

```
// src/main/java/com/placementplanner/web/AdminJobServlet.java (Admin Controller)
@WebServlet("/admin/jobs")
public class AdminJobServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
        request.setAttribute("jobs", jobDAO.findAll());
        request.setAttribute("applications", applicationDAO.findAllForAdmin());
        request.getRequestDispatcher("/WEB-INF/views/admin-jobs.jsp").forward(request, response);
    }
    protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
        String action = request.getParameter("action");
        if ("updateStatus".equalsIgnoreCase(action)) {
            long appId = Long.parseLong(request.getParameter("applicationId"));
            String status = request.getParameter("status"); // SHORTLISTED, INTERVIEW, OFFERED, PLACED
            applicationDAO.updateStatus(appId, status);
            response.sendRedirect(request.getContextPath() + "/admin/jobs?statusUpdated=1");
        } else {
            jobDAO.add(request.getParameter("title"), request.getParameter("company"),
                       request.getParameter("location"), request.getParameter("description"),
                       request.getParameter("eligibility"), Double.parseDouble(request.getParameter("minCgpa")),
                       request.getParameter("deadline"));
            response.sendRedirect(request.getContextPath() + "/admin/jobs?added=1");
        }
    }
}
```

5. Skills Portfolio, Daily Milestones & Modern Frontend Engineering

Candidate qualification tracking is supported through technical skill verification with duplicate elimination and proficiency updating. Students track interview milestones with completion status toggles. ResumeService extracts pure-Java text from PDF/DOCX uploads, evaluating document sections for ATS screening. Client-side interactions in app.js deliver persistent light/dark themes and multi-filter live search.

Technical Skills Portfolio Management src/main/java/com/placementplanner/dao/SkillDAO.java [JAVA]

```
// src/main/java/com/placementplanner/dao/SkillDAO.java (Skill Management)
public void add(long studentId, String skillName, String proficiency) throws SQLException {
    if (hasSkill(studentId, skillName)) return; // Prevent duplicate entries
    String sql = "INSERT INTO skills (skill_id, student_id, skill_name, proficiency, created_at) "
               + "VALUES (SKILLS_SEQ.NEXTVAL, ?, ?, ?, SYSDATE)";
    try (Connection con = DBUtil.getConnection(); PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setLong(1, studentId); ps.setString(2, skillName.trim()); ps.setString(3, proficiency);
        ps.executeUpdate();
    }
}
public boolean updateProficiency(long studentId, long skillId, String proficiency) throws SQLException {
    String sql = "UPDATE skills SET proficiency = ? WHERE student_id = ? AND skill_id = ?";
    try (Connection con = DBUtil.getConnection(); PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setString(1, proficiency.trim()); ps.setLong(2, studentId); ps.setLong(3, skillId);
        return ps.executeUpdate() > 0;
    }
}
```

Preparation Milestones & Status Toggle DAO src/main/java/com/placementplanner/dao/TaskDAO.java [JAVA]

```
// src/main/java/com/placementplanner/dao/TaskDAO.java (Milestones & Completion Toggle)
public boolean updateStatus(long studentId, long taskId, String status) throws SQLException {
    String normalized = "Completed".equalsIgnoreCase(status) ? "Completed" : "Pending";
    String sql = "UPDATE tasks SET status = ? WHERE student_id = ? AND task_id = ?";
    try (Connection con = DBUtil.getConnection(); PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setString(1, normalized); ps.setLong(2, studentId); ps.setLong(3, taskId);
        return ps.executeUpdate() > 0;
    }
}
public void add(long studentId, String title, String dueDate) throws SQLException {
    Date sqlDate = (dueDate != null && !dueDate.isBlank()) ? Date.valueOf(LocalDate.parse(dueDate.trim())) : null;
    String sql = "INSERT INTO tasks (task_id, student_id, title, due_date, status, created_at) VALUES (TASKS_SEQ.NEXTVAL, ?, ?, ?, 'Pending', SYSDATE)";
    try (Connection con = DBUtil.getConnection(); PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setLong(1, studentId); ps.setString(2, title.trim());
        if (sqlDate != null) ps.setDate(3, sqlDate); else ps.setNull(3, Types.DATE);
        ps.executeUpdate();
    }
}
```

Pure-Java Resume Text Extraction & ATS Structural Audit src/main/java/com/placementplanner/service/ResumeService.java [JAVA]

```
// src/main/java/com/placementplanner/service/ResumeService.java (Text Extraction & ATS Audit)
private static void analyzeResumeContent(ResumeInfo info, byte[] bytes, String extension) {
    String text = "docx".equals(extension) ? extractDocxText(bytes) : extractPdfText(bytes);
    int score = 25; // Base score for readable text
    if (Pattern.compile("[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\\.[a-zA-Z]{2,}").matcher(text).find()) score += 15;
    if (Pattern.compile("(?i)\\b(education|academics|b\\.?tech|degree|cgpa)\\b").matcher(text).find()) score += 20;
    if (Pattern.compile("(?i)\\b(technical skills|skills|languages|java|sql)\\b").matcher(text).find()) score += 20;
    if (Pattern.compile("(?i)\\b(projects|academic projects|developed|implemented)\\b").matcher(text).find()) score += 15;
    if (Pattern.compile("(?i)\\b(experience|internship|training|certifications)\\b").matcher(text).find()) score += 10;
    info.setScore(Math.min(100, score));
    info.setStatus(score >= 70 ? Status.PASSED : Status.NEEDS_IMPROVEMENT);
    info.setExtractedSkills(extractSkillsFromText(text));
}
```

Theme Switcher & Live Filter Interactions src/main/webapp/assets/app.js [JS]

```
// src/main/webapp/assets/app.js (Theme Toggle & Client-Side Job Live Search)
function initTheme() {
    const toggleBtn = document.getElementById("themeToggle");
    if (toggleBtn) {
        toggleBtn.addEventListener("click", () => {
            const next = document.documentElement.getAttribute("data-theme") === "dark" ? "light" : "dark";
            document.documentElement.setAttribute("data-theme", next);
            localStorage.setItem("theme", next);
            updateToggleIcon(toggleBtn, next);
        });
    }
}
function filterJobs() {
    const query = searchInput ? searchInput.value.toLowerCase().trim() : "";
    jobCards.forEach(card => {
        const title = (card.dataset.title || "").toLowerCase();
        const matches = (!query || title.includes(query) || card.innerText.toLowerCase().includes(query))
                     && (!locationFilter.value || card.dataset.location.includes(locationFilter.value));
        card.style.display = matches ? "" : "none";
    });
}
```